Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#_07__)

Implementation of Arithmetic Logic Unit (ALU)

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B___

Date performed: __14TH October _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this task we make a program counter that is supporting 4 bit address. In this three modules are required a program counter, instruction memory and testbench so we started it by taking inputs clk that will show the completion of cycle after 5 units (toggles) ,reset that will take the pc to 0000 whenever it becomes 1 and then we make a always block that whenever the pc count changes at the positive edge 0 to 1 or t reset it will be updated then at instruction memory we initializes slot of the memory for each instructions and then we make a testbench.

Code:

```
module program_counter (

   input clk,

   input reset,

   input [3:0] addr,

   output reg [3:0] pc_out

);



always @(posedge clk or posedge reset) begin

   if (reset)

      pc_out <= 4'b0000;

   else

      pc_out <= addr;

end



endmodule

module instruction_memory (

   input [3:0] addr,
```

```
    output reg [3:0] instruction

);



reg [3:0] memory [0:15];



initial begin

    memory[0] = 4'b0001;

    memory[1] = 4'b0010;

    memory[2] = 4'b0011;

    memory[3] = 4'b0100;

end



always @(addr) begin

    instruction = memory[addr];

end



endmodule

module tb_program_counter();



reg clk;

reg reset;

reg [3:0] addr;

wire [3:0] pc_out;
```

```
wire [3:0] instruction;



program_counter pc (

   .clk(clk),

   .reset(reset),

   .addr(addr),

   .pc_out(pc_out)

);



instruction_memory im (

   .addr(pc_out),

   .instruction(instruction)

);



always begin

   #5 clk = ~clk;

end



initial begin

   clk = 0; reset = 0; addr = 0;

   reset = 1; #10 reset = 0;

   #10 addr = 1;

   #10 addr = 2;
```

```
   #10 addr = 3;

   #10 addr = 0;

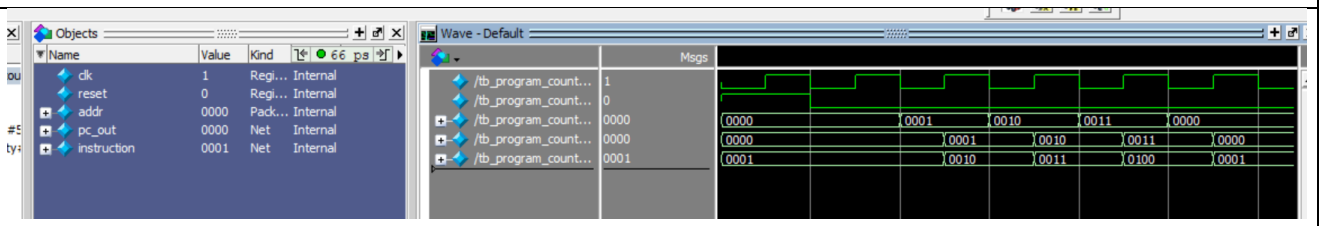
   #20;

end



endmodule
```

Output:



Task 2:

Description:

In this task the requirement is to increment program counter automatically by 1 so in our always block we check if the reset is 1 so update counter 0000 otherwise increment the counter by 1. The whole code will be same and we also add in the instruction memory module that If invalid instruction means if it is out of range here the range of memory is 0 to 15 slots(16) so if it is in that range then we add it otherwise we set it at default 8'b00000000)

Code:

```
module program_counter( clk, reset, addr, pc_out);

input clk, reset;

input [3:0] addr;

output reg [3:0] pc_out;

always @ (posedge clk or posedge reset)
```

```
begin

   if(reset)

     pc_out <= 4'b0000;

   else

    pc_out <= pc_out + 1;

end

endmodule

module instruction_memory(addr, instruction);

input [3:0] addr;

output reg [7:0] instruction;

reg [7:0] memory [0:15];

initial

begin

memory[0]= 8'b00000000;

memory[1]= 8'b00000001;

memory[2]= 8'b00000010;

memory[3]= 8'b00000011;

memory[4]= 8'b00000100;

memory[5]= 8'b00000101;

memory[6]= 8'b00000110;

memory[7]= 8'b00000111;

memory[8]= 8'b00001000;

memory[9]= 8'b00001001;
```

```
memory[10]= 8'b00001010;

memory[11]= 8'b00001011;

memory[12]= 8'b00001100;

memory[13]= 8'b00001101;

memory[14]= 8'b00001110;

memory[15]= 8'b00001111;

end

always @ (addr)

begin

  if ( addr < 16)

    instruction= memory[addr];

  else

    instruction= 8'b00000000;

end

endmodule

module tb_program_counter();

reg clk, reset;

reg [3:0] addr;

wire [3:0] pc_out;

wire [7:0] instruction;

program_counter pc(.clk(clk), .reset(reset), .addr(addr), .pc_out(pc_out));

instruction_memory im(.addr(pc_out), .instruction(instruction));

always begin
```

```
#5 clk= ~clk;

end

initial

begin

clk=0;

reset=0;

addr=0;

reset=1;

#10 reset=0;

#10 addr=1;

#10 addr=2;

#10 addr=3;

#10 addr=0;

#10 addr= 4;

#10 addr= 5;

#10 addr= 6;

#10 addr= 7;

#10 addr= 8;

#10 addr= 9;

#10 addr= 10;

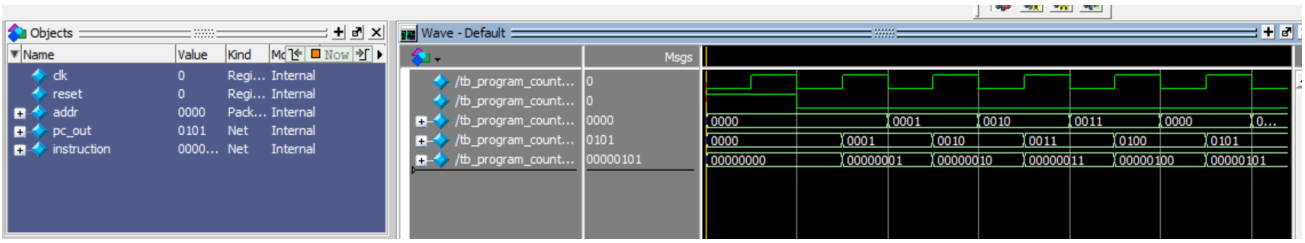
#10 addr= 12;

#10 addr= 16;

end
```

```
endmodule
```

Output:



Task 3:

Description:

In this task we build a 32 bit counter that reset at 0x00000000. On ever clock edge the PC increments by 4 and we use lower 4 bits of the PC as truncated addresss for the small test memory (addr = pc_out[5:2]).the whole code remain same just we did these changings in the code and it become a 32 bit pc counter.

Code:

```
module program_counter( clk, reset, addr, pc_out);

input clk, reset;

input [3:0] addr;

output reg [31:0] pc_out;

always @ (posedge clk or posedge reset)

begin

   if(reset)

     pc_out <= 32'h00000000;
```

```
   else

    pc_out <= pc_out + 4;

end

endmodule

module instruction_memory (

   input [31:0] addr,

   output reg [31:0] instruction

);



reg [31:0] memory [0:15]; // 16 words



initial begin

   memory[0]  = 32'h00000000;

   memory[1]  = 32'h00000001;

   memory[2]  = 32'h00000002;

   memory[3]  = 32'h00000003;

   memory[4]  = 32'h00000004;

   memory[5]  = 32'h00000005;

   memory[6]  = 32'h00000006;

   memory[7]  = 32'h00000007;

   memory[8]  = 32'h00000008;

   memory[9]  = 32'h00000010;

   memory[10] = 32'h00000011;
```

```
    memory[11] = 32'h00000012;

    memory[12] = 32'h00000013;

    memory[13] = 32'h00000014;

    memory[14] = 32'h00000015;

    memory[15] = 32'h00000016;

end



always @(*) begin

    instruction = memory[addr[5:2]];

    // Use bits [5:2] → divide PC by 4

end



endmodule



module tb_program_counter();

reg clk, reset;

reg [3:0] addr;

wire [31:0] pc_out;

wire [31:0] instruction;

program_counter pc(.clk(clk), .reset(reset), .addr(addr), .pc_out(pc_out));

instruction_memory im(.addr(pc_out), .instruction(instruction));

always begin

#5 clk= ~clk;
```

```
end

initial

begin

clk=0;

reset=0;

addr=0;

reset=1;

#10 reset=0;

#10 addr=1;

#10 addr=2;

#10 addr=3;

#10 addr=0;

#10 addr= 4;

#10 addr= 5;

#10 addr= 6;

#10 addr= 7;

#10 addr= 8;

#10 addr= 9;

#10 addr= 10;

#10 addr= 12;

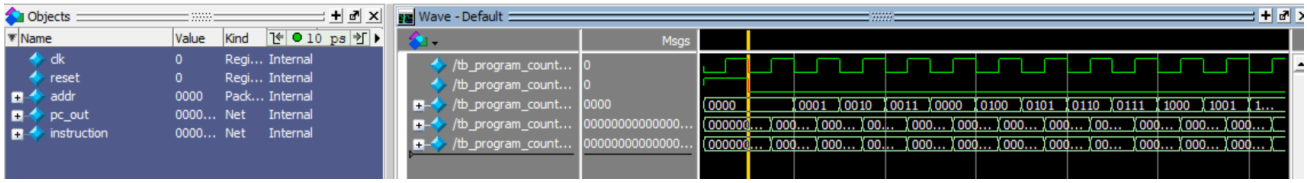
#10 addr= 16;

end



endmodule
```

Output:



Task 4:

Description:

In this task we add an adder module for incrementing the pc by 4 and now the pc uses output of adder to update itself and on reset it becomes 0x00000000. In the instruction memory module it gives 32 bit pc_out and the truncated 4 bit address. With the clock running continuously pc increments by 4 on each cycle the IM module fetches the instructions sequentially.

Code:

```
// Adder module: adds 4 to the current PC

module adder (

   input [31:0] pc_in,

   output [31:0] pc_next

);



assign pc_next = pc_in + 4;



endmodule



module program_counter (

   input clk,

   input reset,
```

```
   output reg [31:0] pc_out,
   output [3:0] addr   // 4-bit truncated address
);

wire [31:0] pc_next;

// Instantiate Adder
adder add4 (
   .pc_in(pc_out),
   .pc_next(pc_next)
);

always @(posedge clk or posedge reset) begin
   if (reset)
      pc_out <= 32'h00000000;   // Reset to base address
   else
      pc_out <= pc_next;       // Update PC using adder output
end

// Truncate PC → lower bits [5:2] give word index
assign addr = pc_out[5:2];

endmodule
```

```
module instruction_memory (

   input [3:0] addr,          // 4-bit address input

   output reg [31:0] instruction

);



reg [31:0] memory [0:15]; // 16 instructions



initial begin

   memory[0]  = 32'h11111111; // 0x00

   memory[1]  = 32'h22222222; // 0x04

   memory[2]  = 32'h33333333; // 0x08

   memory[3]  = 32'h44444444; // 0x0C

   memory[4]  = 32'h55555555; // 0x10

   memory[5]  = 32'h66666666; // 0x14

   memory[6]  = 32'h77777777; // 0x18

   memory[7]  = 32'h88888888; // 0x1C

   memory[8]  = 32'h99999999; // 0x20

   memory[9]  = 32'hAAAAAAAA; // 0x24

   memory[10] = 32'hBBBBBBBB; // 0x28

   memory[11] = 32'hCCCCCCCC; // 0x2C

   memory[12] = 32'hDDDDDDDD; // 0x30

   memory[13] = 32'hEEEEEEEE; // 0x34

   memory[14] = 32'hFFFFFFFF; // 0x38
```

```
    memory[15] = 32'h12345678; // 0x3C

end



always @(*) begin

    instruction = memory[addr]; // Directly use 4-bit addr

end



endmodule

module tb_counter_program();



reg clk, reset;

wire [31:0] pc_out;

wire [3:0] addr;

wire [31:0] instruction;



// Instantiate Program Counter (with adder inside)

program_counter pc (

    .clk(clk),

    .reset(reset),

    .pc_out(pc_out),

    .addr(addr)

);
```

```
// Instantiate Instruction Memory

instruction_memory im (

   .addr(addr),

   .instruction(instruction)

);



// Clock generation

always begin

   #5 clk = ~clk; // Toggle every 5 time units

end



initial begin

   clk = 0; reset = 0;



   // Reset PC

   reset = 1;

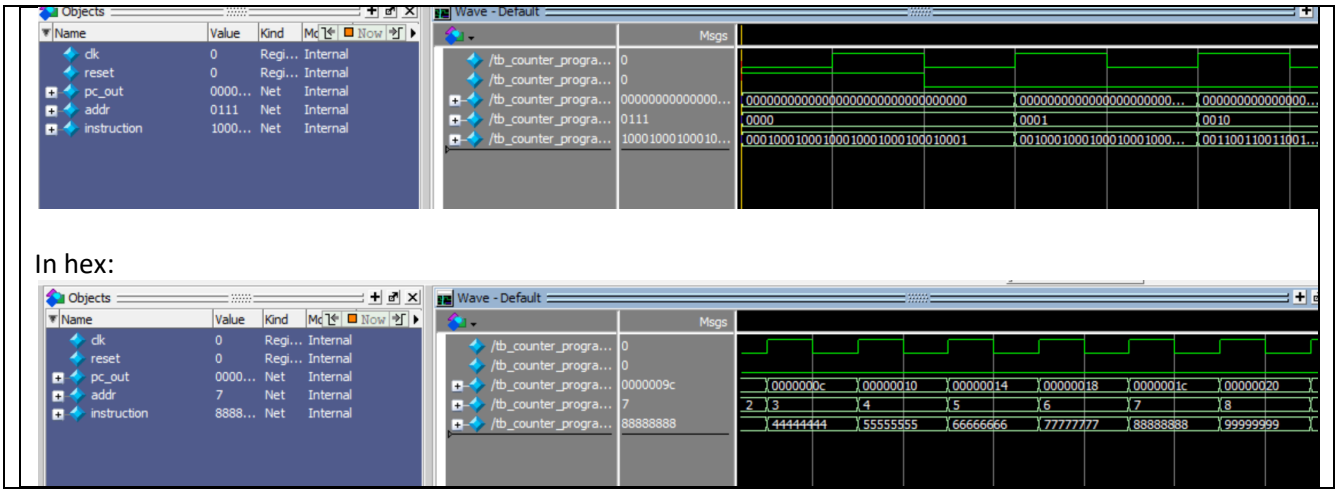
   #10 reset = 0;

end



endmodule
```

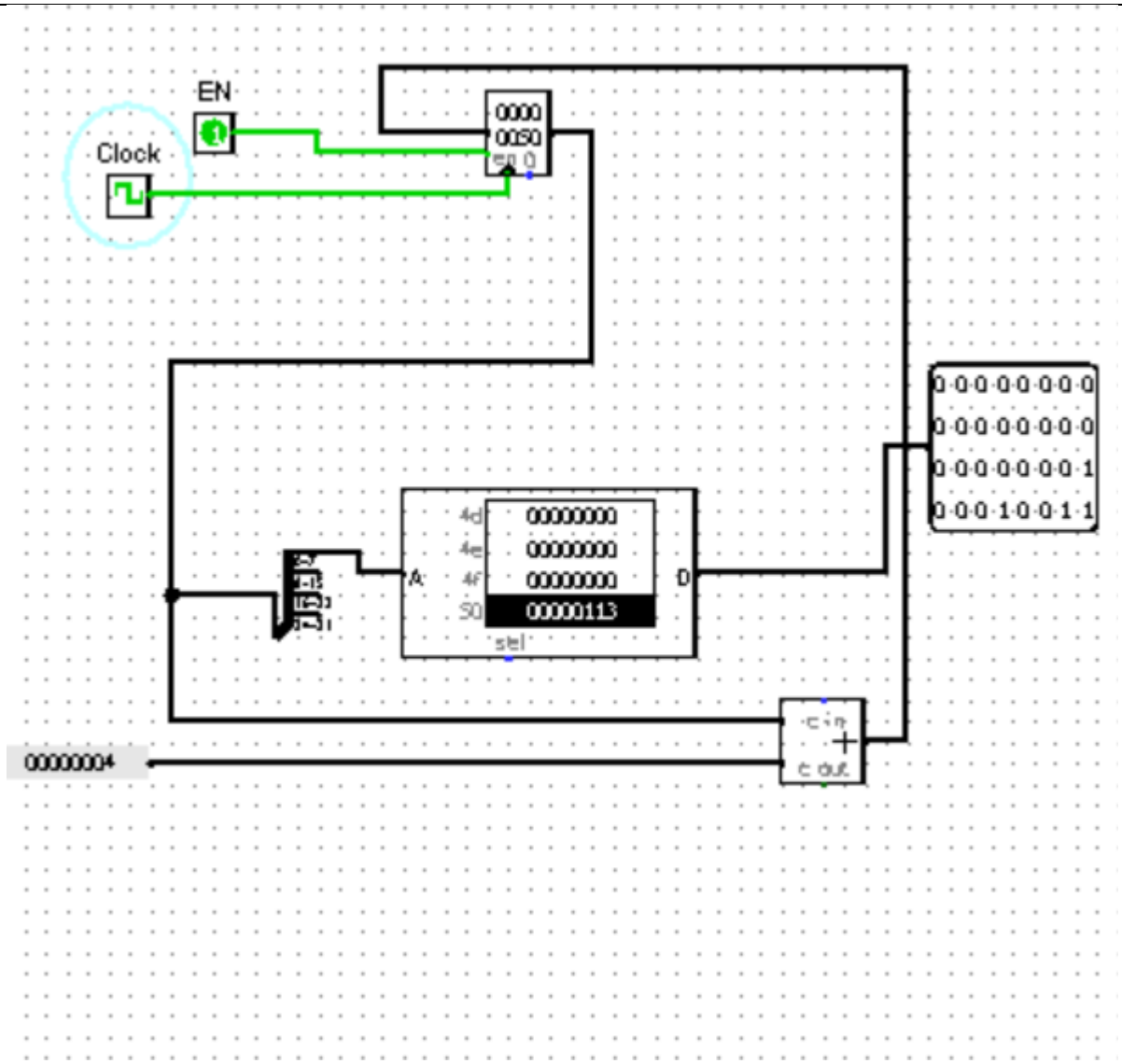
Output:



In hex:

Task 5:

Description:
Here is the 32 bit counter on logisim:
Output:


EN
Clock
0000
0050
en 0
4d
4e
4f
50
00000000
00000000
00000000
00000113
sel
A
D
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1
0 0 0 1 0 0 1 1
c in
c out
00000004